Chiron

L1 Completion, L2 Closure, and Unified Runtime Formalization

Progress Report 13 • 15 Mar 2026 — 31 Mar 2026

Master’s Research Progress Update

Fahad Alothman

Progress Report 13

Title: L1 Completion, L2 Closure, and Unified Runtime Formalization
Reporting Window: 15 Mar 2026 — 31 Mar 2026
Submission Context: Master’s Research Progress Update

Abstract

This reporting window marks the point where Chiron’s methodology-first architecture became materially real across both design time and runtime. The period should not be summarized as “everything is finished.” A more accurate description is that L1 was completed, L2 design-time authoring was closed and hardened, and the unified L1/L2 runtime slice was implemented while L3 step execution remained explicitly deferred.

This distinction matters because it identifies what actually became stable in late March. Chiron moved beyond shell pages and planning documents into a state where methodology structure, work-unit contracts, transition policy, artifact-slot design, and runtime guidance were all represented through coherent surfaces and runtime-backed behavior. The result is a significantly stronger system boundary: design-time methodology authoring and runtime project guidance now reinforce each other instead of existing as separate promises.

The Historical Role of This Window

The most important feature of this window is closure. Earlier periods had already clarified the layered direction of Chiron and had begun defining the correct abstractions. Late March is where those abstractions were pushed through completion and formalization. L1 stopped being an incomplete shell and became a stable methodology overview surface. L2 stopped being a collection of partly designed tabs and became a coherent authoring surface for facts, workflows, state-machine policy, and artifact-slot contracts. Runtime then absorbed that work through the unified L1/L2 slice, making methodology guidance and project execution legible under the same architectural logic.

That makes this period different from the earlier “direction-setting” windows. The project was no longer asking only what the system should be. It was making the agreed design operational.

What Landed Between March 15 and March 31

The early portion of the window included a visible amount of platform and packaging work: desktop runtime bootstrap, preload and IPC contracts, packaged runtime startup, and test-layout standardization. Those changes matter because they improved the execution environment and tightened the delivery baseline. However, they are not the main story of the period.

The main story begins with Story 3.1 completion. During March 17–18, the methodology dashboard, versions shell, author hub, and Work Units (L1) surface converged into a stable design-time layer. CRUD operations for methodology agents, dependency definitions, and work-unit metadata stopped being partial and became part of a coherent authoring path. By March 19, layered service boundaries, design-time/runtime ownership rules, and methodology version authority were explicitly locked into the architecture.

The next movement was Story 3.2 closure. Between March 19 and March 21, the work-unit detail layer (L2) moved from partial contracts into a coherent design-time authoring surface. That closure was then hardened across March 23–27 through additional work on facts, artifact slots, cardinality, and validation behavior. The significance of this is not merely that more tabs existed. It is that each tab now had a clearer contract boundary and a clearer relationship to the methodology model as a whole.

The last movement in the window was the most important runtime step: the unified L1/L2 runtime slice that landed between March 28 and March 29. This is what turned the March work into more than design-time completion. Runtime guidance, active transitions, future candidates, fact state, artifact evidence, and execution detail all became much more credible because they were now backed by a runtime slice designed specifically around L1/L2 semantics rather than older preview/deferred behavior.

L1 Completion: Methodology Topology Becomes Stable

By the end of this window, L1 had crossed the threshold from “good direction” into “stable methodology authoring layer.” The Work Units Overview surface had a clear page purpose, locked shell shape, stable tab structure, synchronized list/canvas selection, and durable action vocabulary. Methodology version ownership, archive-first destructive behavior, and version/work-unit/workflow ownership rules also became much more explicit.

This matters because L1 is the layer where the user builds the project’s method map. If L1 remains unstable, the rest of the methodology model is difficult to trust. Completing L1 therefore means that Chiron gained a stable topological layer for discovering, creating, selecting, and opening methodology work units without mixing that surface with deeper L2 or L3 concerns.

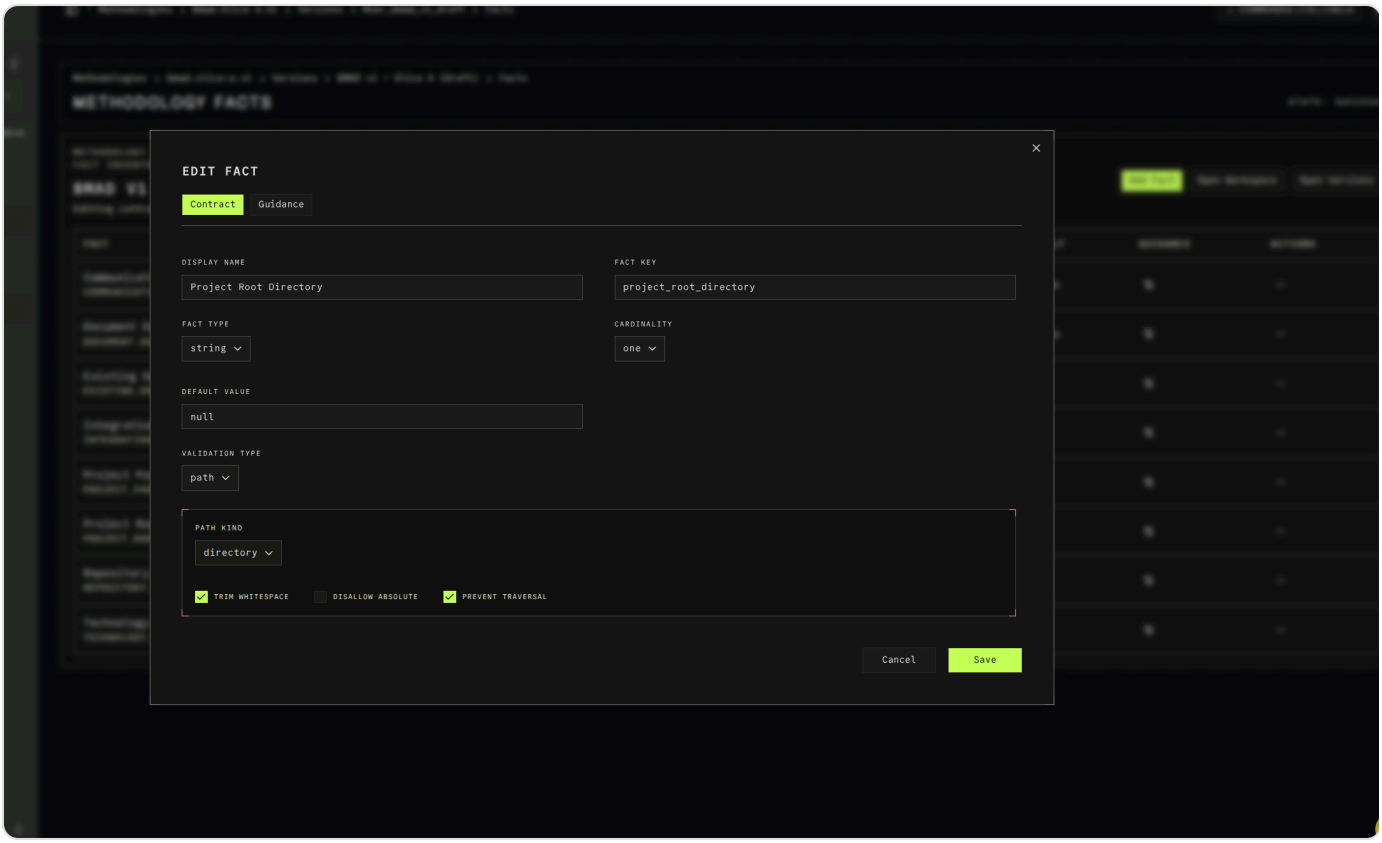

Figure 1. Methodology fact authoring at design time, showing contract fields, cardinality, and validation behavior as first-class methodology concerns.

The methodology fact dialog is best read as adjacent evidence of late-March design-time maturity rather than as the sole proof of L1 topology completion. It shows that methodology-level contract fields such as type, cardinality, default value, and validation policy were being treated explicitly by this point. That matters because Chiron was externalizing more of the project's mental model into durable structures rather than leaving it implicit in chat or human memory.

L2 Closure: Work-Unit Contract Authoring Becomes Coherent

L2 is the layer where Chiron becomes more than a topology browser. It is where each selected work unit receives facts, workflows, lifecycle states, transitions, bindings, and artifact-slot contracts. By the end of this period, this layer had a stable tab model and much stronger ownership boundaries. The selected work unit remained the persistent anchor, while each L2 tab owned its narrower editing concern: Overview summarized and routed, Facts owned work-unit fact contracts, Workflows owned the workflow catalog, State Machine owned transition policy and binding readiness, and Artifact Slots owned durable output contracts.

This is also where the methodology-first value becomes visible in a way that ordinary orchestration tools usually do not provide. The project is not simply creating tasks for agents to

run. It is defining the contract of what a work unit is allowed to mean, how it moves, which outputs matter, and which policies govern readiness.

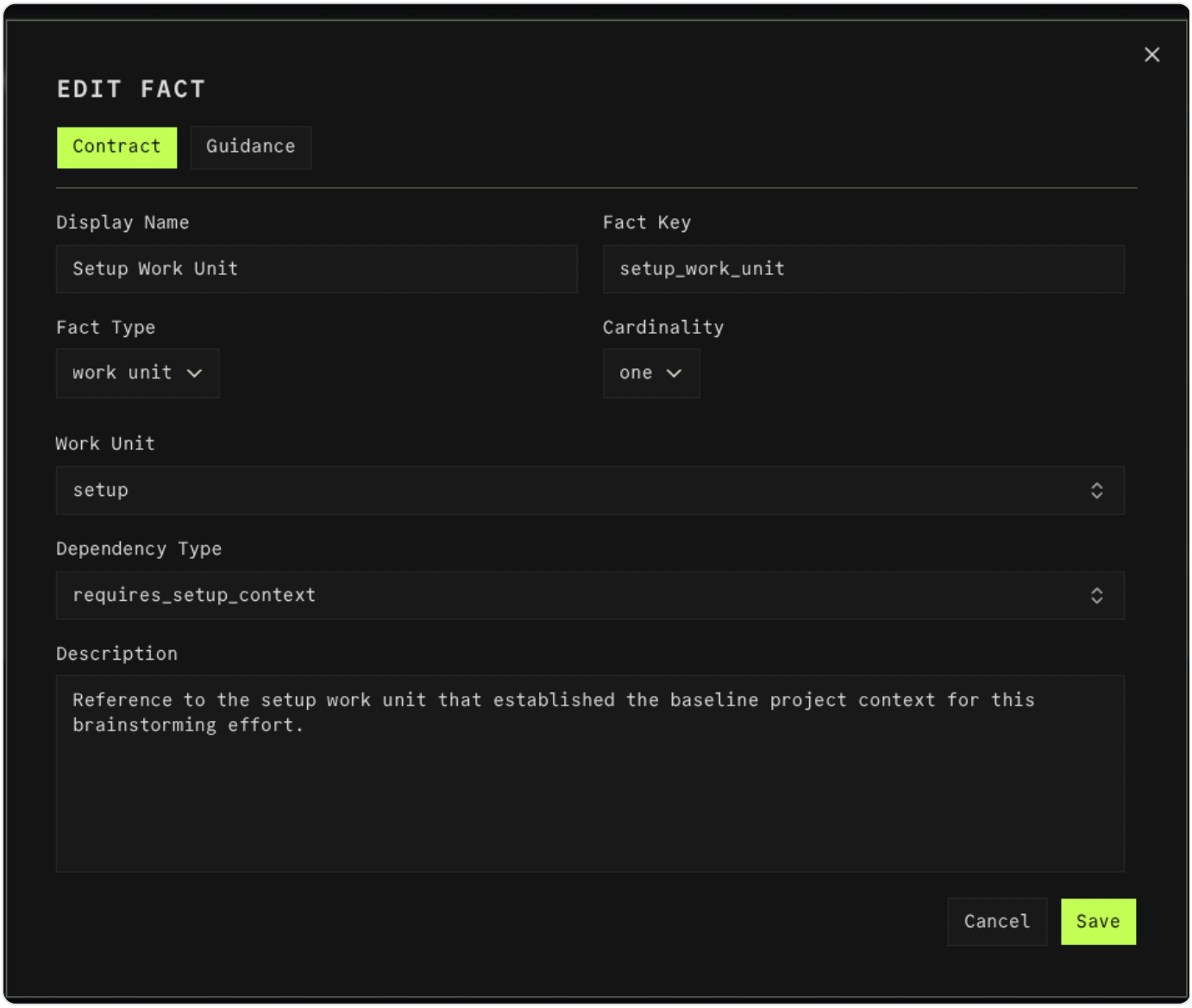

Figure 2. Work-unit fact authoring at design time, showing that work-unit facts can express relationship semantics and work-unit-specific contract rules.

The work-unit fact dialog shows why this layer matters. Work-unit facts do not only store primitive values. They can also carry relationship meaning, dependency type, and work-unit-specific contract behavior. That is part of what lets Chiron move from a workflow-centric model toward a methodology-and-project-centric one.

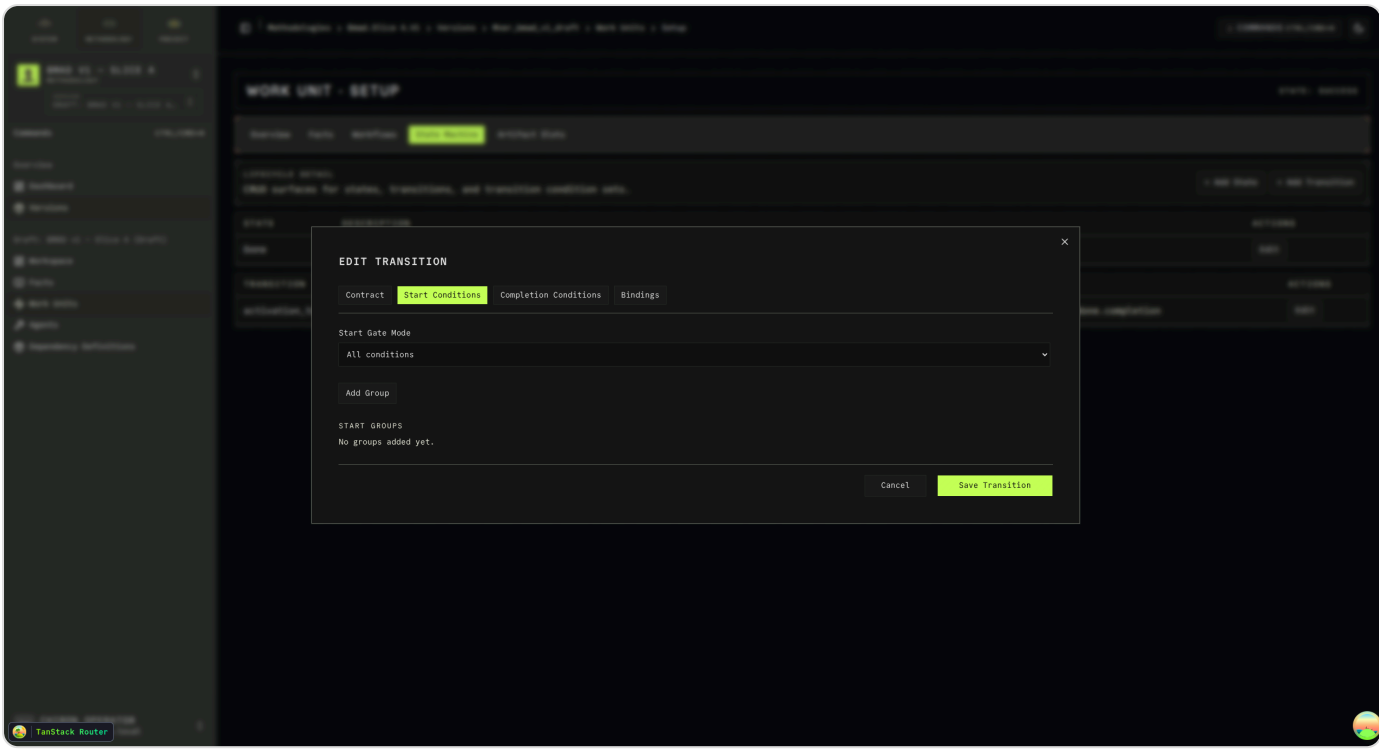

Figure 3. Transition gate authoring inside the L2 State Machine surface, showing grouped start-gate conditions and explicit lifecycle policy ownership.

The transition/state-machine screenshot is arguably the strongest design-time proof in the whole report. It shows that transitions, gate mode, condition grouping, and lifecycle policy are all visible and editable as first-class methodology authoring concerns. This is what turns Chiron into a system that can offload the project's mental model into explicit, inspectable rules rather than leaving progression logic hidden inside agent prompts or developer habit.

Unified L1/L2 Runtime: Guidance Stops Being Speculative

The runtime slice that landed at the end of March is the part that makes this window feel complete rather than merely well documented. The unified L1/L2 runtime implementation replaced deferred or preview-like behavior with runtime-backed guidance, transition execution, work-unit state, fact lineage, and artifact evidence. Crucially, it did this while still explicitly deferring L3 step execution. That boundary is important because it keeps the report honest.

The result is that runtime guidance can now tell the user which work is blocked, which transitions are available, which work unit is active, and which gate conditions are preventing movement. In other words, runtime is no longer loosely mirroring methodology; it is executing against it.

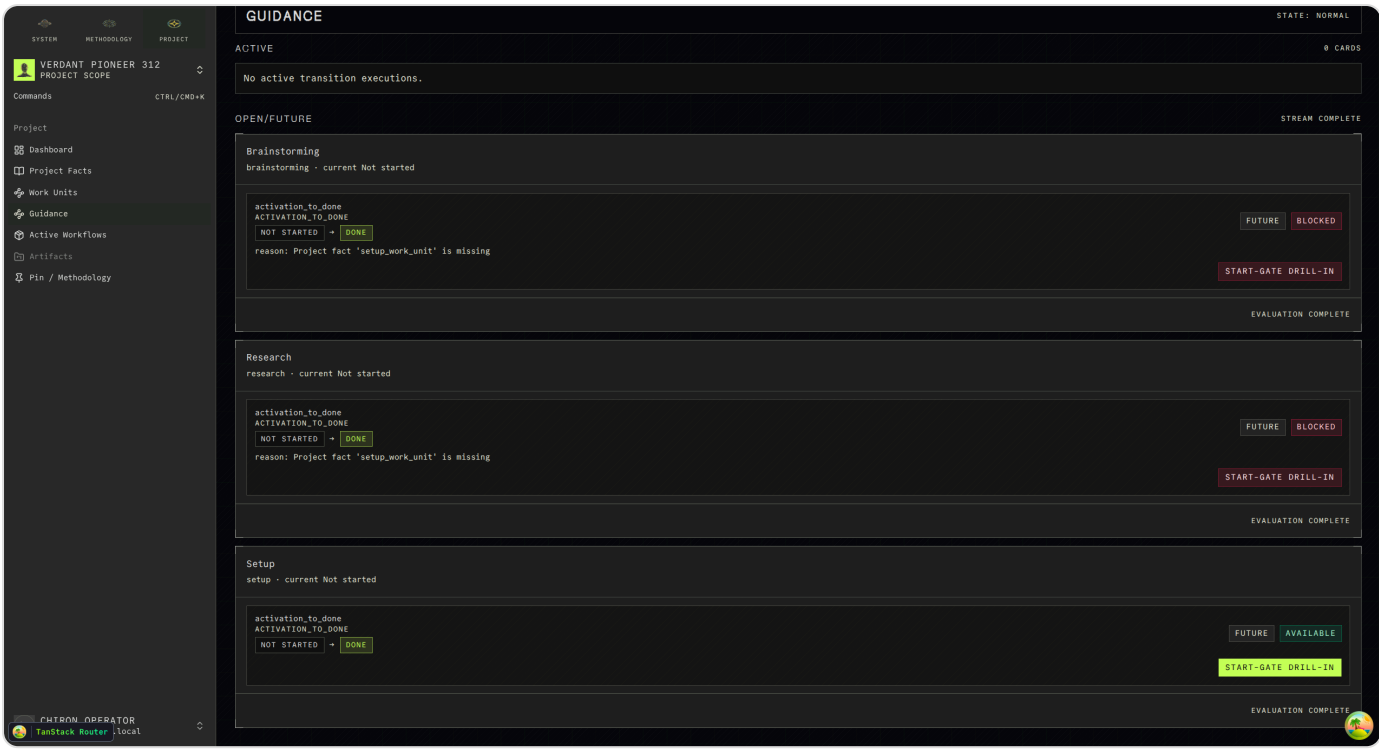

Figure 4. Runtime Guidance page showing blocked and available transitions under project-state and fact-driven gate evaluation.

This screenshot is the clearest runtime representation of Chiron's core value in this period. The system is not merely showing a list of tasks. It is surfacing the current project state as interpreted through methodology-defined transitions and fact conditions. That is exactly the kind of cognitive offloading Chiron is supposed to provide.

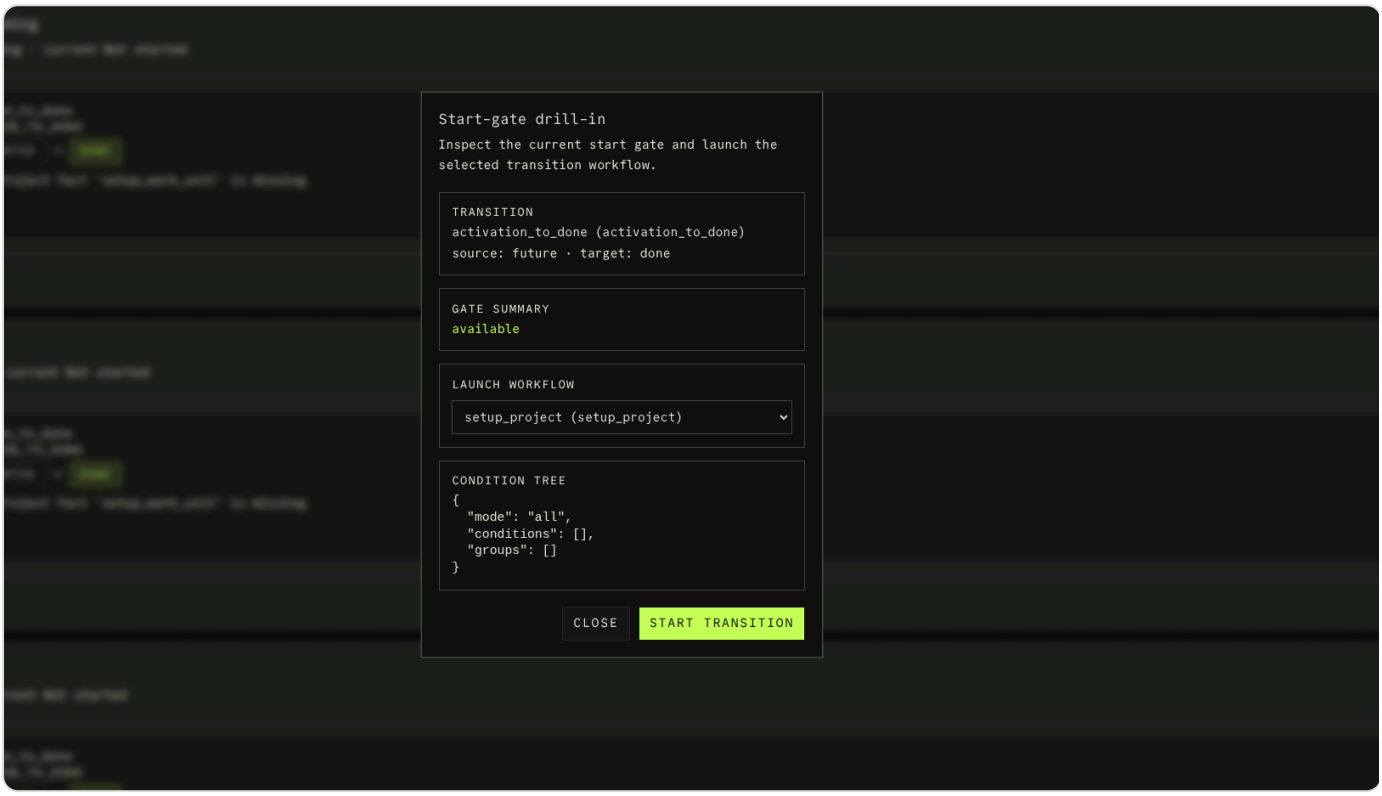

Figure 5. Runtime start-gate drill-in for the Setup transition, showing that available transitions are backed by explicit gate semantics rather than informal workflow navigation.

The start-gate drill-in is a useful complement to the guidance overview because it proves that runtime availability is not arbitrary. Even when the current condition tree is simple, the runtime surface still treats gate evaluation and workflow launch selection as formal policy-backed decisions.

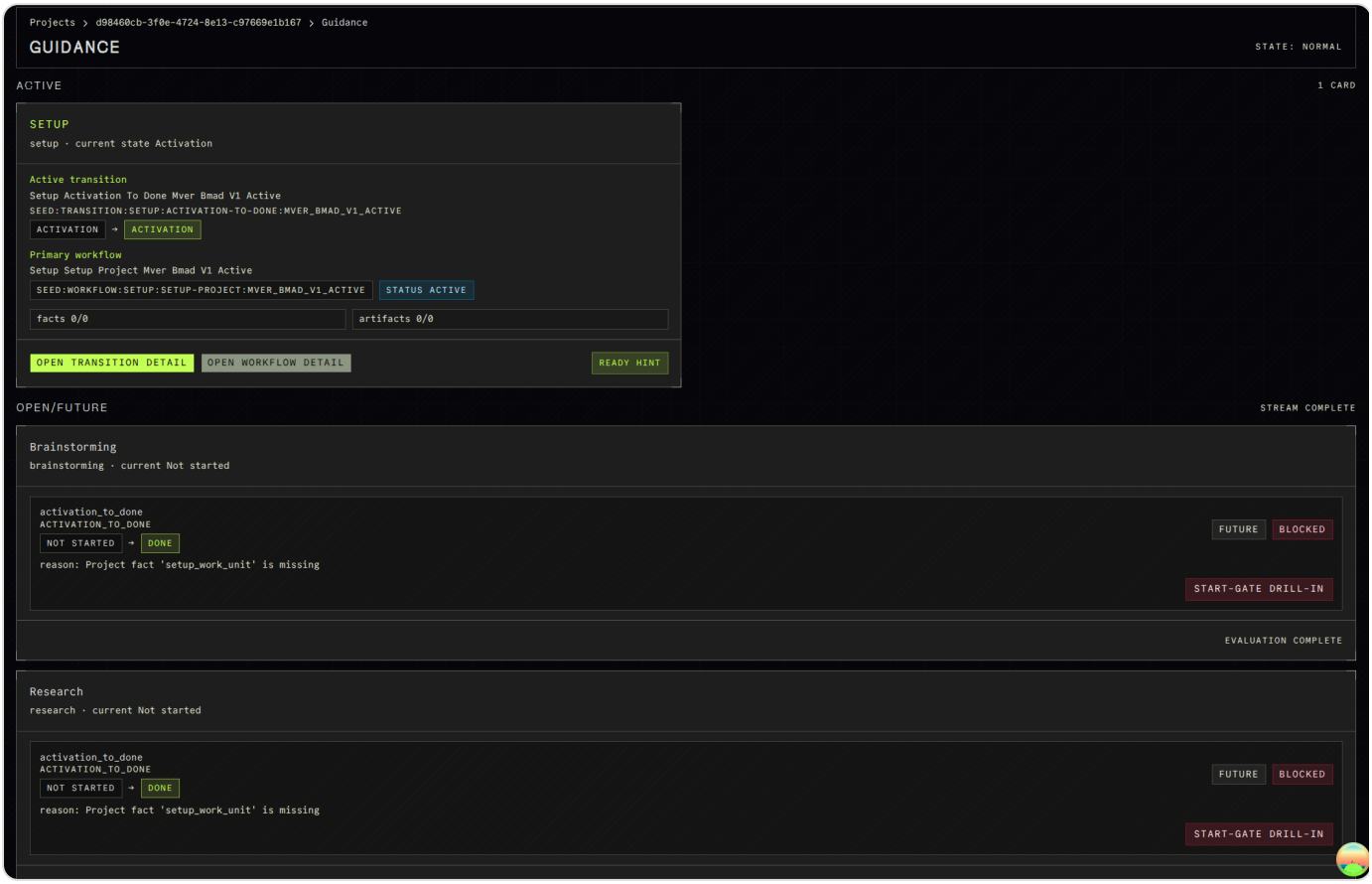

Figure 6. Active transition view in runtime guidance, showing that L1/L2 runtime formalization now tracks active transition state rather than only future candidates.

The active-transition view matters because it shows that guidance is not limited to recommendation. It now also tracks the project's active movement through its methodology-defined transition structure.

What This Window Did Not Finish

This period should not be used to claim that everything from L1 through L3 was completed. The strongest overclaim risk would be to blur the unified L1/L2 runtime slice into a claim that L3 step execution was also finished. The evidence does not support that. The runtime plan for this period explicitly deferred L3 step execution, and the adjacent L3 Form and Agent work belonged more to

planning and next-slice direction than to the main landed accomplishments of this exact reporting window.

That does not weaken the period. It actually sharpens it. The main accomplishment of late March is that Chiron stopped being a promising methodology authoring direction with partial runtime surfaces and became a system where the L1/L2 design-time model and runtime guidance model materially reinforce each other.

Why This Window Matters

This is the window where Chiron becomes much easier to recognize in its own right. It is still true that broader orchestration concerns, isolated worktrees, sandboxing, fuller git integration, signatures, and richer optimizer use remain part of the longer-horizon direction. But by the end of March, the project had made the correct thesis tradeoff: implement the core guidance value first.

That means implementing a system that can hold project structure, lifecycle meaning, facts, artifact contracts, and runtime guidance in a form shared between the user and the agents. In that sense, this window is not only about feature completion. It is about the moment where Chiron's distinctive value becomes operational rather than only conceptual.

Stack Direction at the End of the Window

The technical stack at the end of March reflects that same convergence:

- **Desktop host:** Electron-oriented packaged runtime work still formed part of the local execution path in this window, while the later report set already captures the eventual Tauri-first steady-state direction.
- **Frontend:** React + TanStack Router + shadcn/ui
- **Backend/API:** TypeScript services with Hono + oRPC-aligned runtime surfaces [1], [2]
- **Execution/control plane:** Effect-first service layering with methodology/runtime boundary discipline [3]
- **Data layer:** Drizzle ORM with methodology-first and runtime-lineage persistence [4]
- **Git/workspace seam:** still limited mainly to artifact truth, lineage, and staleness rather than full worktree orchestration [5]
- **Runtime/tooling:** Bun + Turborepo [6]

The important point is that the stack in this window is serving a methodology-first execution model. It is not yet trying to prove the full long-horizon orchestration picture at the same time.

Conclusion

Progress Report 13 should be read as the late-March closure window in which L1 was completed, L2 design-time authoring was closed and hardened, and the unified L1/L2 runtime slice was implemented. That combination made Chiron's methodology-first architecture materially real across both design-time and runtime surfaces while still keeping L3 step execution out of scope for the period.

Its deepest significance is not just that more surfaces existed by the end of March. It is that Chiron's core value — guidance and shared project understanding — became substantially more executable and inspectable. That is why this window deserves to stand as its own report.

References

[1] Hono Authors, "Hono," Documentation, 2026. [Online]. Available: https://hono.dev/
[2] oRPC, "oRPC Documentation," 2026. [Online]. Available: https://orpc.unnoq.com/
[3] Effect Authors, "Effect Documentation," 2026. [Online]. Available: https://effect.website/
[4] Drizzle Team, "Drizzle ORM Documentation," 2026. [Online]. Available: https://orm.drizzle.team/
[5] simple-git, "simple-git," GitHub repository, 2026. [Online]. Available: https://github.com/simple-git-js/simple-git/
[6] Turborepo Authors, "Turborepo Documentation," 2026. [Online]. Available: https://turbo.build/repo/docs